

High-Performance Implementation of Multigrid Solver for Lattice QCD

面向 Lattice QCD 传播子求解的高性能多重网格实现

汇报人 张鑫

中国科学院近代物理研究所

2026-10-11 15:05–15:20

项目 [https:](https://github.com/zhangxin8069/PyQCU)

[//github.com/zhangxin8069/PyQCU](https://github.com/zhangxin8069/PyQCU)

镜像 [https:](https://gitee.com/zhangxin8069/PyQCU)

[//gitee.com/zhangxin8069/PyQCU](https://gitee.com/zhangxin8069/PyQCU)

文档 [https://gitee.com/](https://gitee.com/zhangxin8069/PyQCU/tree/main/docs)

[zhangxin8069/PyQCU/tree/main/docs](https://gitee.com/zhangxin8069/PyQCU/tree/main/docs)

日志 [https://gitee.com/](https://gitee.com/zhangxin8069/PyQCU/tags)

[zhangxin8069/PyQCU/tags](https://gitee.com/zhangxin8069/PyQCU/tags)

性能结论统一采用 `git tag test27`: 66 个精确单元、132 条 side record

Wilson 与 Clover: 先固定可持续求解的细网格算子

Wilson Dslash

$$\begin{aligned}(H\psi)_x &= \sum_{\mu=0}^3 \left[(1 - \gamma_\mu) U_{x,\mu} \psi_{x+\hat{\mu}} \right. \\ &\quad \left. + (1 + \gamma_\mu) U_{x-\hat{\mu},\mu}^\dagger \psi_{x-\hat{\mu}} \right], \\ D_W &= (m_0 + 4)I - \frac{1}{2}H \\ &= (m_0 + 4)(I - \kappa H), \\ \kappa &= \frac{1}{2m_0 + 8}.\end{aligned}\tag{1}$$

来源: docs/report_numerical_linear_algebra_lattice_qcd_20260911.md; pyqcu/dslash/_wilson.py; pyqcu/dslash/_clover.py

Clover 改进 (当前 $u_0 = 1, c_{\text{sw}} = 1$)

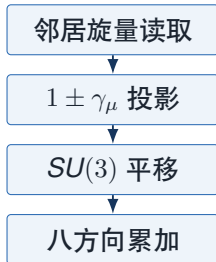
$$\begin{aligned}Q_{\mu\nu} &= P_{\mu\nu} + P_{\nu,-\mu} + P_{-\mu,-\nu} + P_{-\nu,\mu} \\ C_{\mu\nu} &= Q_{\mu\nu} - Q_{\mu\nu}^\dagger \\ T_\rho &= -\frac{\kappa c_{\text{sw}}}{8u_0} \sum_{\mu < \nu} \gamma_\mu \gamma_\nu C_{\mu\nu} \\ A_\rho &= I_\rho + T_\rho\end{aligned}\tag{2}$$

原始 Dslash: 八个邻居上的矩阵自由稀疏作用

数据流与入口语义

- 每格点读取四个正向和四个反向 neighbors;
- $1 \pm \gamma_\mu$ 将四分量 spinor 投影为两个分量;
- 执行 $SU(3)$ 平移并累加八个方向贡献。

原始入口返回裸核 H ; 带单位项入口返回 $I - (\kappa/u_0)H$ 。



性能本质: 最近邻模板天然 bandwidth-bound; $4 \rightarrow 2$ spin projection、 $SU(3)$ 重用和尾置时空维决定有效带宽。

来源: `cpp/cuda/qcu/src/wilson_dslash.cu:55-218;`
`pyqcu/testing/qcu/single_function_common.py:325;` 508 报告优化 1-4

奇偶消元：外层 Krylov 维度减半，解仍可精确重建

块矩阵与 Schur 补

$$D_{\text{pc}} = \begin{pmatrix} A_e & -\kappa H_{eo} \\ -\kappa H_{oe} & A_o \end{pmatrix} \quad (3)$$

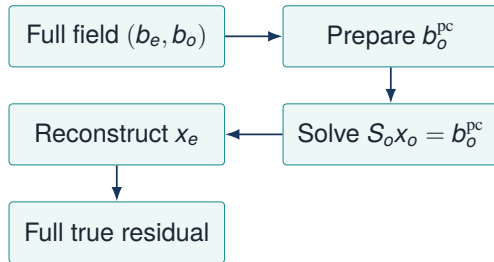
$$S_o = A_o - \kappa^2 H_{oe} A_e^{-1} H_{eo} \quad (4)$$

求解与重建

$$b_o^{\text{pc}} = b_o + \kappa H_{oe} A_e^{-1} b_e \quad (5)$$

$$S_o x_o = b_o^{\text{pc}} \quad (6)$$

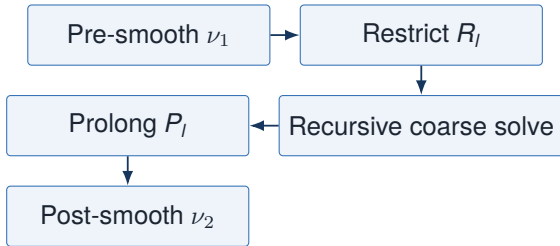
$$x_e = A_e^{-1} (b_e + \kappa H_{eo} x_o) \quad (7)$$



MultiGrid: 低模粗空间与高频平滑器互补

四个核心对象

- 近零模: $A_l B_l \approx 0$
- 局部正交: $P_l^\dagger P_l = I$
- 转移对: $R_l = P_l^\dagger$
- Galerkin 粗算子: $A_{l+1} = R_l A_l P_l$



$$\mathbf{e} = \mathbf{e}_{\text{high}} + \mathbf{e}_{\text{low}}, \quad A_l B_l \approx 0 \quad (8)$$

来源: report_pyqcu_mg_operator_construction_20260830.tex:80-148
lattice_multigrid.h:11-25

预条件 Bi-CGStab: 粗层求解的核心算法

Algorithm 1 Preconditioned Bi-CGStab

$$r^{(0)} = b - Ax^{(0)}$$

$$\tilde{r} = r^{(0)}$$

for $i = 1, 2, \dots$

$\rho_{i-1} = \tilde{r}^\dagger r^{(i-1)}$; **if** $\rho_{i-1} = 0$ **fail**

if $i = 1$ **then** $p^{(1)} = r^{(0)}$

else

$$\beta_{i-1} = \frac{\alpha_{i-1}\rho_{i-1}}{\omega_{i-1}\rho_{i-2}}$$

$$p^{(i)} = r^{(i-1)} + \beta_{i-1}(p^{(i-1)} - \omega_{i-1}v^{(i-1)})$$

solve $M\hat{p} = p^{(i)}$; $v^{(i)} = A\hat{p}$

$$\alpha_i = \rho_{i-1} / (\tilde{r}^\dagger v^{(i)})$$

$$s = r^{(i-1)} - \alpha_i v^{(i)}$$

if $\|s\|/\|b\| \leq \varepsilon$ **then** $x^{(i)} = x^{(i-1)} + \alpha_i \hat{p}$; **stop**

solve $M\hat{s} = s$; $t = A\hat{s}$

$$\omega_i = t^\dagger s / (t^\dagger t)$$

$$r^{(i)} = s - \omega_i t$$

$$x^{(i)} = x^{(i-1)} + \alpha_i \hat{p} + \omega_i \hat{s}$$

end for

来源: 算法语义: report_multigrid_quda_pyqcu_20260928.tex; 实现:
lattice_{wilson, clover}_bistabcg.h

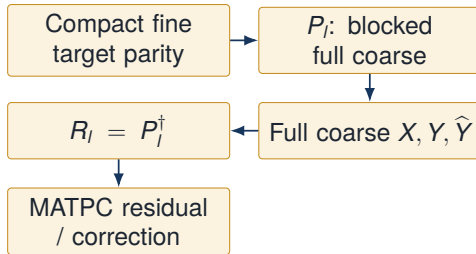
Strict 特化: $D = X + H$ 与 full coarse 资产

层级分解

$$\begin{aligned} D_I &= X_I + H_I \\ \hat{D}_I &= X_I^{-1} D_I \\ D_{I+1} &= R_I \hat{D}_I P_I \end{aligned} \quad (9)$$

运行资产: $X, X^{-1}, Y^{f/b}, \hat{Y}^{f/b}, V$ 。粗层保留 full geometry; 细层使用 compact target parity。

来源: `apply_multigrid_strict.cu:1499-1615, 2926-3430`
`_strict_galerkin.py:428-719`



粗层 X/Y : onsite block、方向链接与 $\hat{Y}$ 顺序

X 与 Y 的构造

$$E = 2n_v, \quad X_\ell, Y_{\ell,\mu}^{\text{f/b}} \in \mathbb{C}^{E \times E} \quad (10)$$

$$X_C(X) = \sum_{x \in \mathcal{A}_X} V_X^\dagger C V_X \quad (11)$$

$$Y_\mu^{\text{f}}(X) = -\kappa \sum_{x \in \mathcal{A}_X} (A V_X)^\dagger U_\mu V_{X+\hat{\mu}} \quad (12)$$

$$Y_\mu^{\text{b}}(X - \hat{\mu}) = -\kappa \sum_{x \in \mathcal{A}_X} V_X^\dagger U_\mu^\dagger A V_{X-\hat{\mu}} \quad (13)$$

$AV = C^{-1}V$ (Clover-PC); raw path 取 $AV = V$ 。

raw action 与预条件链接

$$D_c z(X) = X(X)z(X) + \sum_{\mu} \left[Y_\mu^{\text{f}}(X)z(X + \hat{\mu}) + Y_\mu^{\text{b}}(X - \hat{\mu})^\dagger z(X - \hat{\mu}) \right] \quad (14)$$

$$\hat{Y}_\mu^{\text{f}}(X) = X^{-1}(X)Y_\mu^{\text{f}}(X) \quad (15)$$

$$\hat{Y}_\mu^{\text{b}}(X - \hat{\mu}) = Y_\mu^{\text{b}}(X - \hat{\mu})X^{-\dagger}(X) \quad (16)$$

$$M_p^c x_p = x_p - \hat{H}_{pq} \hat{H}_{qp} x_p \quad (17)$$

PC dslash 只应用 $\hat{Y}$ hopping; backward 使用 $X - \hat{\mu}$ 的 storage block, 左右矩阵顺序不可交换。

来源: report_multigrid_quda_pyqcu_20260913.tex:116-168, 190-204; 交叉实现: apply_multigrid_strict.cu; _strict_galerkin.py

V-cycle 在里、FGMRES 在外：MG 预条件的算法闭环

Algorithm 2 Recursive MG preconditioner

Input : $A_l, r_l, \{S_{\text{pre}}, S_{\text{post}}, P_l, R_l\}$
if $l = L - 1$ **then** $z_l \leftarrow A_l^{-1} r_l$
else
 $y_l \leftarrow S_{\text{pre}}^{\nu_1}(r_l)$
 $r_{l+1} \leftarrow R_l(r_l - A_l y_l)$
 $e_{l+1} \leftarrow M_{l+1}^{-1} r_{l+1}$
 $z_l \leftarrow y_l + P_l e_{l+1}$
 $z_l \leftarrow S_{\text{post}}^{\nu_2}(z_l)$
end if

$$x^{(k+1)} = x^{(k)} + z^{(k)} \quad (18)$$

来源: `apply_multigrid_strict.cu:2926-3430, 4579-4683; report_multigrid_quda_pyqcu_20260928.tex`

Algorithm 3 Flexible FGMRES

$r_0 = b - Ax^{(0)}, v_1 = r_0 / \|r_0\|$
for $j = 1, \dots, m$
 $z_j = M^{-1} v_j; w = Az_j$
 for $i = 1, \dots, j$
 $h_{ij} = v_i^\dagger w; w = w - h_{ij} v_i$
 end for
 $h_{j+1,j} = \|w\|; v_{j+1} = w / h_{j+1,j}$
 apply Givens rotations to H_j
end for
 $x^{(k+1)} = x^{(k)} + Z_j y$

CUDA-C++ 优化：从寄存器、流、显存延伸到 MPI

细层 kernel

$1 \pm \gamma$ 四到二投影;
 $SU(3)$ 重用; 尾置 $xyzt$
合并访存。

粗层延迟

device scalar; 融合标
量/向量更新; CUDA
Graph; fused coarse
solve。

Setup / 显存

colored/batched
Galerkin; workspace
cap; persistent
hierarchy; SHA256
cache。

通信

rank-local halo;
nonblocking vector
halo; remote correction
融合。

关键判断： 细层是 bandwidth-bound，粗层是 latency-bound；两类瓶颈必须分别优化，单点 kernel 调优不足以获得端到端收益。

来源: `lattice_clover_multigrid.h:24-30, 755-789, 3584-3806; _strict_cache.py:32-1074; report_multigrid_optimized_20260927.tex`

对比口径：66 个精确单元覆盖设备、精度、层级和 trace

矩阵协议

V100 单卡；双 P100 $2 \times 1 \times 1 \times 1$
c64 / c128；BiCGStab / MG-2 /
MG-3

trace on/off

1 cold + 2 warmup + 5 steady

请求 72 combined；精确 66
combined、132 side；
6 个双 P100 c64 大格显式替代。

来源：配置/输入哈希 66/66；真残差 66/66；trace-on MG 曲线 44/44

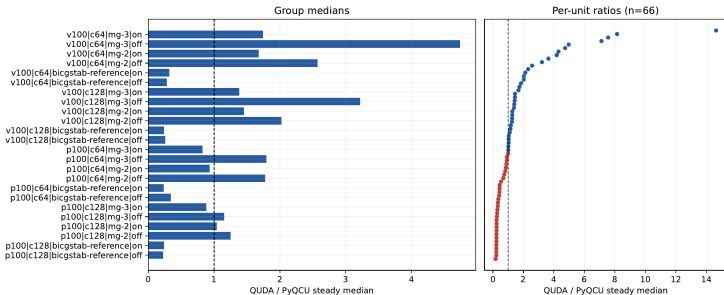


图 1: $R = t_{\text{QUDA}}/t_{\text{PyQCU}}$, $R > 1$ 表示 PyQCU 更快；正式性能使用 trace-off。

总体结果：MG 专项优势明确，但整体中位仅为 1.025

1.025
66 个精确单元总中位

35/44
MG-2/3 有利

21/22
MG trace-off 有利

0/22
BiCGStab reference

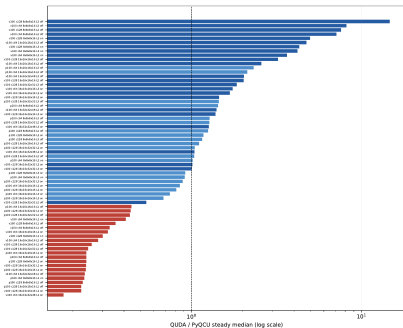


图 2: 66 个精确单元逐点比值。

来源: report_multigrid_quda_pyqcu_20260928.tex, test27/units.csv

PyQCU MultiGrid

test27 · 2026-09-28

12/18

设备与精度趋势：V100 明显领先，P100 在 trace-off 有利

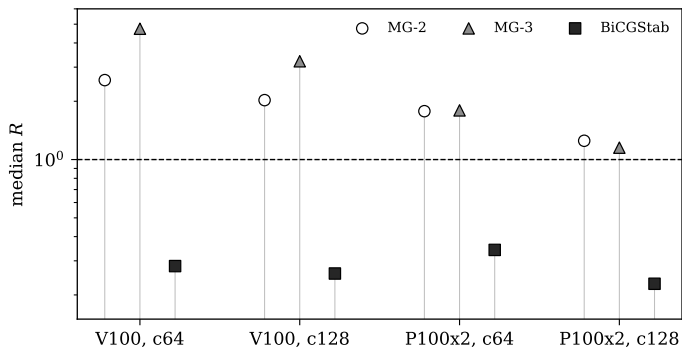


图 3: trace-off 分组中位比值。

来源: test27/overlap_matrix.csv; report_multigrid_quda_pyqcu_20260928.tex

trace-off 分组中位

设备/精度	MG-2	MG-3	BiCG
V100/c64	2.57	4.74	0.28
V100/c128	2.03	3.22	0.26
P100/c64	1.78	1.79	0.34
P100/c128	1.25	1.15	0.23

V100 c128 MG-3 最大单点 14.63; 也存在 0.54 慢点, 因此不能只看极值。

trace-on 阶段与显存: coarse solve 是主要记录成本



图 4: trace-on 逐层阶段分解。

来源: test27/stages.csv; report/mg_memory.csv; capacity_substitutions.csv

PyQCU MultiGrid

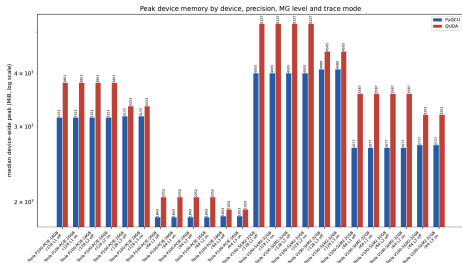


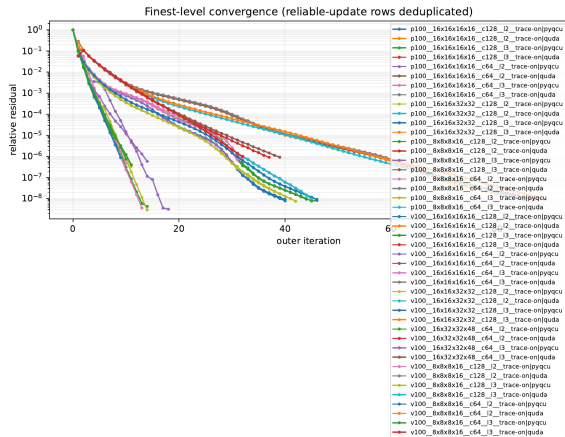
图 5: device-wide 峰值显存, 非进程独占。

PyQCU 最大 device-wide 观测显存 11907.3 MiB;
QUDA 为 23063.3 MiB;
双 P100 c64 大格 6 项显式替代为 16⁴。

test27 · 2026-09-28

14/18

正确性与下一步：真残差过门，粗层同步仍是主战场



正确性

PyQCU 最大真残差 7.40×10^{-7} ;
QUDA 最大真残差 7.97×10^{-7} ;
均低于 5×10^{-6} 。

下一步

- coarse solve 与同步融合
- device-aware MPI / 持久 request
- setup 流式化与显存压缩
- 国产平台端到端验证

图 6：递推残差仅用于趋势；最终门限为 full-operator true residual。

来源：report_multigrid_quda_pyqcu_20260928.tex; report_multigrid_optimized_20260927.tex:327-342

参考实现与最短复现接口

参考实现

- quantum-mg: Evan Weinberg / NVIDIA
- QUDA: multigrid, staging, coarse operator
- DD- α AMG: Matthias Rottmann
- 本地源码: `refer/git-rep/**`

复现顺序

1. `source ./env.sh`
2. `bash ./build.sh & bash ./install.sh`
3. `run_strict_fast.py --fail-fast`
4. `strict_mpi_solve_probe.py`
5. `bench_mg_matrix_full.py --list`

平台状态: CUDA 已实测; CPU 部分; DCU/CANN 历史或仿真; TileLang 为 smoke; 历史大规模测试见下一页。

来源: 正式运行应以 `main` 最新入口和 `provenance` 为准

先前成果：508 报告完成 64–1024 进程大格测试

1024
进程/线程规模

3.12×
Wilson 报告加速

7.19×
Clover 报告加速

≈2048 GiB
64 卡池峰值估算

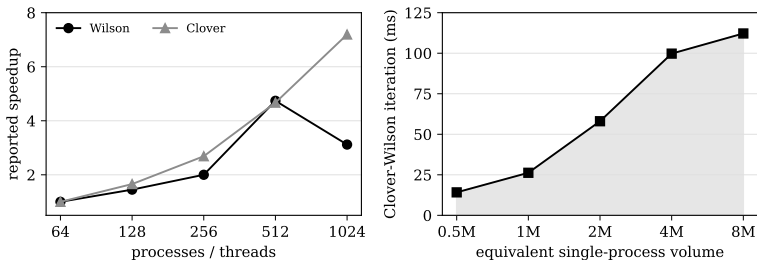


图 7：左为 TEST8 c64 强扩展；右为报告弱扩展差异汇总。

数据来源：508 报告 TEST8 c64 与 TEST9/汇总表；1024 为进程/线程规模，不是千卡 GPU 实测。

来源：508 报告 §1.5–1.6；早期 DCU 路径成果，不与 test27 Strict-MG 速度比混合

谢谢

PyQCU Strict MultiGrid

下一步：粗层同步、通信重叠和 setup 显存做成可复验收益。

`https://github.com/zhangxin8069/PyQCU`

数据基线: `gittagtest27`

Q & A